

2. Gestión de eventos

2.1. Eventos: tipos

Los eventos de JavaScript son acciones que ocurren sobre determinados elementos. Algunas de esas acciones están causadas por el usuario, mientras que otras pueden ser causadas por otros motivos, como la carga de una página, la recepción completa de un archivo o la finalización de un temporizador. Estos eventos están definidos en la especificación del DOM.

Algunos de los eventos más comunes son:

- Eventos de ratón. **click**, **mousedown**, **mouseup**, **dblclick**, **mouseenter**, etcétera.
- Eventos de foco. **focus** (el elemento recibe el foco), **blur** (el elemento pierde el foco).
- Eventos de teclado. **keydown** y **keyup**.
- Eventos de formulario. **submit**, envío de formulario.
- Eventos de documento. **DOMContentLoaded**, que se produce cuando el documento HTML se ha cargado por completo.

! IMPORTANTE

El código JavaScript se ejecuta en la página HTML cuando se carga su etiqueta **<script>** correspondiente. Si queremos ejecutar un programa que haga referencia a algún elemento del DOM, es necesario esperar a que sea cargado. Así, los *scripts* definidos dentro de **<head>** no pueden hacer referencia a ningún nodo que cuelgue de **document.body**, ya que **<body>** todavía no se habrá cargado.

Para solucionar este problema, hay dos opciones:

- Situar la etiqueta **<script>** con el código del programa al final del elemento **<body>**.
- Poner el programa principal dentro de la función manejadora del evento **DOMContentLoaded**. En este caso, el programa se ejecutará cuando se cargue completamente el documento.

Mediante JavaScript, es posible detectar dichos eventos y ejecutar funciones en respuesta a ellos. A estas funciones se las conoce como **manejadores de eventos** (*event handlers*, en inglés).

La primera opción para escribir un manejador de eventos es escribirlo directamente sobre el elemento HTML. Para ello, se utilizan los atributos **onEVENTO**, donde **EVENTO** es el evento que se quiera capturar. Por ejemplo:

```
<button id="boton" onclick="saludar()">Pulsa el botón</button>
```

```
<script>
function saludar() {
    alert("Hola Mundo!");
}
```

```
</script>
```

La segunda opción consiste en hacerlo desde JavaScript:

```
<button id="boton">Pulsa el botón</button>

<script>
  function saludar() {
    alert("Hola Mundo!");
  }

  document.getElementById("boton").onclick = saludar;
</script>
```

! IMPORTANTE

Como puede verse en los anteriores ejemplos:

- En HTML hay que indicar el código que queremos ejecutar realizando la llamada a la función [esto es, incluyendo los paréntesis].
- En JavaScript hay que indicar el nombre de la función [esto es, sin incluir los paréntesis].

La última opción es la más recomendable, ya que, a diferencia de las anteriores, permite asociar más de un manejador de eventos a un elemento. Consiste en utilizar la función `addEventListener`. A continuación, se muestra un ejemplo de uso:

```
<button id="boton">Pulsa el botón</button>

<script>
  function saludar1() {
    alert("Hola Mundo!");
  }

  function saludar2() {
    alert("Hola otra vez!");
  }

  let boton = document.getElementById("boton");
  boton.addEventListener("click", saludar1);

  // Se puede añadir un segundo manejador de eventos, que se ejecutará tras
  el anterior
  boton.addEventListener("click", saludar2);

</script>
```

Esta opción permite también utilizar la función `removeEventListener` para eliminar un manejador de eventos:

```
<button id="boton">Pulsa el botón</button>

<script>
  function saludar1() {
    alert("Hola Mundo!");
  }
  let boton = document.getElementById("boton");

  // Añadir manejador de eventos
  boton.addEventListener("click", saludar1);

  // Código...

  // Eliminar manejador de eventos
  // Hay que indicar de nuevo el evento y la función asociada
  boton.removeEventListener("click", saludar1);
</script>
```

Las funciones manejadoras de eventos tienen acceso a la siguiente información:

- **this**. Hace referencia al elemento sobre el que se produce el evento (excepto si se realiza la llamada desde HTML, en cuyo caso hace referencia a `window`).
- Pueden definir un **parámetro**, que será el objeto evento con información sobre el evento producido. Algunas de las propiedades de este objeto son:
 - **type**. El tipo de evento producido.
 - **currentTarget**. El elemento asociado al manejador de eventos.
 - **target**. El elemento sobre el que se produce el evento.

A continuación, puedes ver un ejemplo:

```
<button id="boton">Pulsa el botón</button>

<script>
  function saludar(evento) {
    console.log(`El evento se ha producido sobre el objeto ${this}`);
    console.log(`Evento de tipo ${evento.type} sobre el elemento ${evento.currentTarget}.`);
  }

  let boton = document.getElementById("boton");
  boton.addEventListener("click", saludar);
</script>
```

La diferencia entre `event.target` y `event.currentTarget` se explicará con más detalle en la siguiente sección.

! IMPORTANTE

Si se desea pasar el evento a la función manejadora desde el atributo HTML, hay que utilizar la variable **event**: `<button id="boton" onclick="saludar(event)">Pulsa el botón</button>`.

2.2. Propagación de eventos

Dado que el DOM es jerárquico, la mayoría de los eventos se **propagan** a lo largo de la **cadena de ancestros**. Esto quiere decir que, si se añade un manejador de eventos a un elemento, el evento «**sube**» por la jerarquía de elementos y puede **ser detectado** por los ancestros de dicho elemento.

Por ejemplo, dada la siguiente estructura de HTML, si se produce un evento **click** en el párrafo, dicho evento podrá ser detectado también en el elemento **<div>**:

```
<div>
  <p>Párrafo</p>
</div>
```

Esto tiene una consecuencia: es posible **añadir un manejador de eventos** a un elemento **contenedor**, de manera que, si se produce un evento en **cualquiera de sus descendientes**, el evento será detectado por dicho contenedor. Por ejemplo:

```
<div onclick="alert('Click!')">
  <p>Párrafo 1</p>
  <p>Párrafo 2</p>
  <p>Párrafo 3</p>
</div>
```

Si pulsamos en cualquiera de los párrafos, se mostrará una alerta, ya que, aunque el evento se produzca en el párrafo, este se propaga y llega a su elemento padre, en este caso el **<div>**, que es el que tiene definido el manejador.

Una vez comprendido este mecanismo, podemos ver la diferencia entre **target** y **currentTarget**: en el caso del ejemplo, **target** será el **párrafo** (elemento sobre el que se produce el evento) y **currentTarget** será el **<div>** (elemento que tiene definido el manejador de evento).



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A02_01.html**, con el código de ejemplo.

2.3. Delegación de eventos

La propagación de eventos posibilita la utilización del patrón de **delegación de eventos**. Su filosofía consiste en **asignar un único manejador de eventos** al elemento **contenedor**, en lugar de asignar multitud de eventos similares a un gran número de elementos. La propiedad **event.target** se puede utilizar para hacer un **filtrado de elementos** sobre los que es necesario actuar o para **acceder a la información** de dichos elementos.

La delegación de eventos es muy útil para trabajar con **componentes** o **widgets**. De esta manera, el **código JavaScript solo se cargaría una vez** y se podrían añadir los componentes que se desearan **únicamente** mediante **HTML**. A continuación, veremos un ejemplo para implementar componentes de tipo **acordeón**.



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A02_02.html**, con el código de ejemplo.

Mediante el uso de la **delegación** de eventos y los atributos del **dataset**, podemos crear tantos componentes acordeón como queramos utilizando **simplemente HTML**, sin escribir ninguna línea de JavaScript. Este estilo de creación de componentes recibe el nombre de **estilo declarativo**, ya que **no necesita código de programación**, sino simplemente **lenguaje de marcado**.



IMPORTANTE

La mayoría de librerías CSS, como **Bootstrap**, ofrecen la opción declarativa para utilizar sus componentes.

2.4. Comportamiento por defecto

El navegador define algunos comportamientos por defecto asociados a algunos eventos. Los más interesantes son:

- **submit** (formularios). El navegador envía los datos del formulario a la URL correspondiente si se produce el evento **submit** de dicho formulario. Ese evento puede producirse al hacer clic en un botón de envío o al pulsar la tecla **Enter** en un campo.
- **click** (enlaces). El navegador abandona la página actual y carga la URL indicada en el enlace.

Estos comportamientos se pueden impedir si se utiliza un manejador de eventos y se llama a la función `event.preventDefault()`. Por ejemplo:

```
<a href="https://google.es" onclick="accion1(event)">Ir a Google</a>
<a href="https://google.es" onclick="accion2(event)">Ir a Google</a>

<script>
  function accion1(evento) {
    console.log("Se ejecuta el código y se va a Google");
  }

  function accion2(evento) {
    evento.preventDefault();
    console.log("Se ejecuta el código y NO se va a Google");
  }
</script>
```

! IMPORTANTE

Incluye siempre el atributo **type** en los botones de los formularios, tanto los de tipo `<button>` como los de tipo `<input>`. Si no lo haces, se comportarán como botones de tipo `type="submit"`, por lo que activarán el evento **submit** cuando se pulse sobre ellos.

Si quieres que los botones no activen el evento **submit** de manera automática, utiliza `type="button"`.

2.5. Objetos manejadores de eventos

`addEventListener` permite definir una función manejadora de eventos y también un objeto manejador de eventos. La sintaxis es la misma: únicamente deberemos pasar como parámetro un objeto que implemente el método `handleEvent`.

Para comprender la utilidad de esta opción, partiremos de un ejemplo de aplicación de tipo *SPA*, *single-page application* o aplicación de una sola página. En este tipo de aplicaciones, es habitual trabajar con una **API de servidor** que nos devuelva los datos en un formato tipo **JSON**. Nuestra tarea como desarrolladores *front-end* será recibir esos datos y **presentarlos al usuario a través de una interfaz**. Dicha interfaz nos debe permitir **interactuar con esos datos**, que normalmente estarán almacenados en **objetos**, por lo que debemos encontrar la manera de establecer una **conexión** entre la **representación HTML** del objeto y los **datos** (y posiblemente métodos) de dicho objeto.

Tomaremos como ejemplo la gestión de un listado de personas: para cada elemento de la lista mostraremos sus datos junto con un botón para **editar** y otro para **borrar**. Así, si tenemos cuatro personas, tendremos cuatro botones para editar y cuatro para borrar. Por tanto, necesitamos establecer una **conexión** entre cada **botón** y la **persona** a la que está asociada.

Si utilizamos una **función manejadora de eventos**, no tenemos manera fácil de establecer esa conexión: podríamos utilizar atributos `id` junto con atributos del **dataset** para establecer la asociación, de manera parecida a como hemos hecho en el ejemplo anterior de acordeón. Pero, dado que la aplicación almacena el

listado de personas en un *array*, habría que realizar una búsqueda en dicho *array* cada vez que se pulsara en un botón. En lugar de ello, vamos a optar por utilizar **objetos manejadores de eventos**, que permiten definir **información de contexto** que utilizaremos para pasar el objeto **persona** asociado.

En primer lugar, definimos la estructura de datos que utilizaremos en el ejemplo, que será muy sencilla:

```
let personas = [
  {
    nombre: "Ana",
    apellido: "López"
  },
  {
    nombre: "Inés",
    apellido: "García"
  }
];
```

A continuación, definimos objetos que definan la función **handleEvent** y que se utilizarán como **prototipo** de los objetos manejadores de eventos:

```
let editarPersona = {
  handleEvent: function(evento) {
    alert(`Editando datos de ${this.datosPersona.nombre} ${this.datosPersona.apellido}`);
  }
}

let borrarPersona = {
  handleEvent: function(evento) {
    alert(`Borrando datos de ${this.datosPersona.nombre} ${this.datosPersona.apellido}`);
  }
}
```

Tanto **editarPersona** como **borrarPersona** son objetos. Ambos implementan la función **handleEvent**, por lo que pueden utilizarse como **manejadores de eventos** en **addEventListener**. Como van a ser utilizados como prototipos, hacen uso de **this** para hacer referencia a **datosPersona**, que almacenará el objeto de tipo **persona** asociado.

Por último, a partir de la lista de datos se construirá el interfaz HTML. Para cada **persona** se creará:

- Una **estructura HTML** utilizando las funciones del DOM para visualizar los **datos** y mostrar los **botones** de edición y borrado.
- Un **objeto manejador de eventos** para el botón de edición que tenga acceso al objeto **persona** correspondiente a través de la propiedad **datosPersona**. Se construirá a partir del prototipo **editarPersona**.
- Un **objeto manejador de eventos** para el botón de borrado que tenga acceso al objeto **persona** correspondiente a través de la propiedad **datosPersona**. Se construirá a partir del prototipo **borrarPersona**.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A02_03.html**, con el código de ejemplo.